

RELAZIONE TECNICA DI PROGETTO

BOMBERMAN

Ottimizzazione del Codice con gli LLM

Realizzato da: **Lorenzo Gaglioti, Imram Numan Ali, Thomas Ghini**

1. Descrizione del Progetto e Utilizzo del Software

Il progetto è una versione del classico gioco **Bomberman** per terminale, scritta in C++ con l'aiuto della libreria **ncurses**. Il flusso del gioco è controllato dalla classe `Gamemaster`, che gestisce il menù iniziale navigabile con mouse o tastiera, la sessione di partita, la classifica dei record salvata su file di testo e la schermata finale.

Compatibilità Multiplatforma (Windows e Linux): Il gioco è stato progettato per funzionare nativamente su entrambi i sistemi operativi. Per garantire la piena compatibilità su Windows, abbiamo modificato la configurazione di compilazione (`CMakeLists.txt`) istruendo il sistema a linkare dinamicamente la libreria `pdccurses`. Inoltre, all'interno del codice sorgente (es. `Schermo.hpp`), abbiamo utilizzato le direttive del preprocessore (`#ifdef _WIN32`) per mappare e tradurre correttamente funzioni specifiche del sistema, sostituendo ad esempio la lettura dei click del mouse di Linux (`getmouse`) con l'implementazione nativa equivalente di Windows (`nc_getmouse`).

La partita si svolge su una griglia fissa di 20 righe $\times$ 40 colonne. Il giocatore comanda il personaggio (@) attraverso **5 livelli**, rompendo casse fragili (%), raccogliendo power-up ed eliminando tutti i nemici (0) per passare al livello successivo.

- **Movimento (WASD):** Con W, A, S, D il player si sposta sulla griglia. Il metodo `Player::play()` controlla prima sulla mappa se la casella è vuota (' '), bloccando il movimento se ci sono muri o casse.
- **Piazzamento Bombe (X):** Premendo X si deposita una bomba (B). La bomba attende 3 secondi in stato `COUNTING`, poi esplode per 200 millisecondi in stato `EXPLODING` generando fiamme a croce (*).
- **Power-Up (Item):** Rompendo una cassa c'è la possibilità di trovare oggetti a terra: **Range (R)** (bombe con raggio +2 per 3 cariche), **Speed (S)** (passo raddoppiato per 5 secondi) e **Life (L)** (+1 punto vita).
- **Livelli e Punteggio:** Si può navigare tra i livelli con N (Next) e P (Prev). Eliminati tutti i nemici, il livello viene superato (+500 punti e +120 secondi extra). I punteggi finali vengono salvati sul file `Classifica.txt`.

2. Come è Stato Diviso il Lavoro nel Gruppo

Abbiamo suddiviso il progetto in tre moduli ben distinti per lavorare in parallelo senza creare conflitti:

Membro	Classi e File	Compiti Svolti
Lorenzo Gaglioti	Player.hpp/.cpp Bomb.hpp/.cpp Enemy.hpp (+ t1,t2,t3) Lists.hpp	Ho sviluppato tutto il comportamento del Player (movimento, collisioni, vite, cooldown bombe, potenziamenti), delle Bombe (macchina a stati, timer chrono), dei Nemici polimorfici (3 tipologie) e le liste concatenate per gestire le entità in memoria.
Imram Numan Ali	Mappa.hpp/.cpp GestoreLivelli.hpp/.cpp Item.hpp/.cpp	Ho sviluppato la griglia 20x40, la generazione casuale di muri e casse tramite generatore LCG, la colorazione dei blocchi a schermo in ncurses, la gestione dei Power-up e la lista doppiamente concatenata per scorrere i livelli.
Thomas Ghini	Menu.hpp/.cpp Classifica.hpp/.cpp Schermo.hpp Tempo/Punti.hpp	Ha realizzato i menù grafici con gestione dei click del mouse, il resize responsive della finestra, il timer della partita con <ctime> e la lettura/scrittura con ordinamento della classifica su file.

3. PARTE 1: LORENZO GAGLIOTI - Player, Bombe, Nemici e Gamemaster

3.1 Principali Sfide Tecniche e Come Sono State Affrontate

Nello sviluppo della mia parte (Player, Bombe, Nemici e gestione nel Gamemaster), le sfide principali sono nate dai requisiti didattici dell'esame: **divieto assoluto di usare la libreria STL** (niente `std::vector`, `std::list` o `std::string`) e obbligo di **gestire manualmente la memoria con `new` e `delete`**, evitando categoricamente memory leak e segmentation fault.

1. Gestione Entità tramite Liste Concatenate Singole (Lists.hpp)

Dato che bombe innescate, mostri ed item variano continuamente di numero, ho implementato delle liste concatenate singole native con le struct `bomb_node` (`bnode`), `enemy_node` (`enode`) e `item_node` (`inode`). L'inserimento avviene in testa in tempo $O(1)$.

```
// Esempio di inserimento e pulizia manuale in testa
bnode tmp = new bomb_node{Bomb(y, x), bhead};
bhead = tmp;
```

Quando una bomba conclude l'esplosione o un nemico viene colpito, il nodo viene scollegato e deallocato. La pulizia totale avviene con `Gamemaster::svuotaEntita()` al cambio livello.

2. Separazione tra Matrice Logica della Mappa e Rendering a Schermo

Nelle prime versioni, quando entità si muovevano, scrivevo direttamente @ o X nella matrice griglia della mappa. Questo causava problemi: muovendosi sopra una cassa, il nemico la cancellava. Ho risolto separando i ruoli: la matrice griglia memorizza solo gli elementi dell'ambiente, mentre Player e Nemici vivono con coordinate separate. Il Gamemaster controlla prima le collisioni con `mappa->getCella(y, x)` e poi disegna le entità "sopra" lo schermo tramite `stampaEntita()` prima del refresh.

3. Cooldown delle Bombe senza Bloccare il Gioco

Per evitare che il giocatore creasse un bomb-spamming tenendo premuto 'X', serviva un timer. Usare `time(NULL)` non era sufficiente perché ha precisione solo al secondo. Ho risolto usando `std::chrono::steady_clock` nel Player: il metodo `canPlaceBomb()` calcola la differenza in millisecondi e concede una bomba solo se sono passati almeno 1000 ms.

4. Polimorfismo per la Gestione dei Nemici

Per gestire 3 tipi di nemici senza riempire il loop principale di switch-case, ho creato la classe base astratta `Enemy` con il metodo `virtual void move() = 0`. Ho poi derivato tre classi: `Enemy_t1` (movimento ortogonale), `t2` (più veloce) e `t3` (movimento diagonale). Nel Gamemaster salvo tutti i nemici nella stessa lista invocando il polimorfismo.

3.2 Utilizzo degli LLM per Ottimizzare il Codice

Durante lo sviluppo della mia parte ho utilizzato gli LLM come supporto per la scrittura e l'ottimizzazione di algoritmi chiave. Approccio critico: Non ho accettato passivamente le risposte dell'LLM. Spesso il modello proponeva costrutti moderni della STL come `std::vector`. Ho quindi guidato l'LLM con prompt restrittivi per rispettare rigorosamente i vincoli didattici di C++ manuale.

1.1 Caso d'Uso 1: Rimbalzo Fisico e Movimento Diagonale (`Enemy_t3::move`)

Obiettivo: Gestire il rimbalzo sugli assi X e Y in modo separato, per rendere il movimento del nemico più fluido e realistico.

Contesto iniziale: La classe `Enemy_t3` rappresenta il nemico più veloce. Nella mia implementazione iniziale il codice verificava l'intera cella diagonale di arrivo in blocco: se occupata, invertiva simultaneamente le direzioni (`dirY = -dirY; dirX = -dirX;`). Il nemico rimbalzava all'indietro a 180 gradi invece di scivolare sulle pareti.

Prompt inviato all'LLM:

"Sto sviluppando un nemico in C++ che si muove in diagonale in una griglia 2D. Il mio codice attuale inverte sia dirX che dirY contemporaneamente, tornando indietro sui suoi passi. Come posso separare il rimbalzo sull'asse X e sull'asse Y in modo pulito e realistico?"

Risposta dell'LLM:

"Per ottenere una riflessione realistica stile 'biliardo', verifica le collisioni sui due assi in modo indipendente: 1. Controlla un ostacolo muovendoti solo in verticale (newY, position.x): se sì, inverti solo dirY. 2. Controlla un ostacolo muovendoti solo in orizzontale: se sì, inverti solo dirX. 3. Se dopo queste correzioni la casella diagonale finale risulta libera, aggiorna la posizione."

Codice Prima dell'Ottimizzazione:

```
if (diff >= 150) {
    int nextY = position.y + dirY;
    int nextX = position.x + dirX;

    // Controlla solo la casella diagonale
    if (mappa->getCella(nextY, nextX) == ' ') {
        position.y = nextY;
        position.x = nextX;
    } else {
        // Inverte entrambi: torna a 180 gradi
        dirY = -dirY;
        dirX = -dirX;
    }
}
```

Codice Finale Ottimizzato (src/Enemy_t3.cpp):

```
if(diff >= 150){
    int newY = position.y + dirY;
    int newX = position.x + dirX;
    // Rimbalzo indipendente asse Y
    if (mappa->getCella(newY, position.x) != ' ') {
        dirY = -dirY;
        newY = position.y + dirY;
    }
    // Rimbalzo indipendente asse X
    if (mappa->getCella(position.y, newX) != ' ') {
        dirX = -dirX;
        newX = position.x + dirX;
    }
    if (mappa->getCella(newY, newX) == ' ') {
        position.y = newY; position.x = newX;
    }
}
```

Differenze e Valutazione Critica: L'ottimizzazione suggerita ha risolto perfettamente il calcolo del movimento. Dividendo la collisione sui due assi, il nemico rimbalza a 90 gradi lungo muri e casse continuando a pattugliare la mappa in modo aggressivo, senza incastrarsi. L'LLM aveva anche proposto di memorizzare una matrice di float per calcolare traiettorie con seni e coseni, ma **ho respinto questa complessità** preferendo l'aritmetica discreta, che è infinitamente più efficiente e pulita per ncurses.

1.2 Caso d'Uso 2: Cooldown delle Bombe senza Bloccare il Gioco

Obiettivo: Evitare il bomb-spamming garantendo un intervallo minimo di 1 secondo tra due bombe piazzate.

Prompt inviato all'LLM:

"Nel mio gioco le bombe possono essere piazzate velocemente premendo ripetutamente X, creando uno spamming che sovraccarica il loop di aggiornamento. Come posso introdurre un cooldown preciso usando std::chrono in modo che venga rispettato un intervallo di 1 secondo, senza usare thread aggiuntivi?"

Risposta dell'LLM:

"Usa std::chrono::steady_clock per memorizzare l'ultimo timestamp di piazzamento della bomba. Confronta il timestamp corrente con quello salvato e consenti una nuova bomba solo se la differenza supera 1000ms. Questo evita il blocco del thread e mantiene il loop di gioco fluido."

Codice Prima dell'Ottimizzazione:

```
// Prima non c'era controllo di cooldown
bool Player::canPlaceBomb() {
    return true; // always allowed
}

void Player::placedBomb() {
    // Nessun timestamp salvato
    last_bomb_time = std::chrono::
        steady_clock::now();
}
```

Codice Finale Ottimizzato (Player.cpp):

```
bool Player::canPlaceBomb() {
    auto now = std::chrono::steady_clock::
        now();
    long long diff = std::chrono::
        duration_cast<std::chrono::milliseconds
        >
        (now - last_bomb_time).count();

    return diff > 1000; // 1 second
        cooldown
}

void Player::placedBomb() {
    last_bomb_time = std::chrono::
        steady_clock::now();
}
```

Differenze e Valutazione Critica: L'uso di `steady_clock` fornisce un timing ad alta risoluzione senza bloccare il thread principale (che ascolta gli input ncurses). Il codice rimane estremamente semplice, leggibile e rispetta perfettamente il vincolo di non usare container STL. **Sintesi dei Risultati per la mia parte:** L'interazione con l'LLM ha permesso di ottimizzare le prestazioni del calcolo dei movimenti, preservando integralmente l'architettura manuale C++ richiesta dai vincoli didattici.

4. PARTE 2: IMRAM NUMAN ALI - Mappa, Livelli, Oggetti e Collisioni

4.1 Principali Sfide Tecniche e Come Sono State Affrontate

Mi sono occupato principalmente di sviluppare la gestione della Mappa di gioco, il sistema di avanzamento dei livelli, il funzionamento degli Items e il meccanismo per rilevare le collisioni.

1. Generazione Procedurale della Mappa

Il problema era creare mappe casuali simili al gioco originale, che risultassero diverse a ogni avvio, ma sempre giocabili (senza percorsi totalmente bloccati). Ho implementato un generatore di numeri casuali (LCG) usando una variabile `seme_casuale`. Ho strutturato la mappa con un array 2D `griglia[20][40]`, in cui una logica di `switch-case` decide la quantità di casce e muri in base alla difficoltà (Livelli da 1 a 5).

2. Scorrimento Livelli tramite Lista Bidirezionale

Dato il divieto rigoroso dell'uso della libreria standard di C++, per salvare e navigare i livelli in avanti e indietro ('n' e 'p'), ho dovuto ideare una struttura dati ad hoc. Ho inserito i dati della Mappa in una **lista doppiamente concatenata** gestita dalla classe `GestoreLivelli`.

```
// src/GestoreLivelli.cpp
temp->next = nuovoLivello;
nuovoLivello->prev = temp;
nuovoLivello->next = NULL;
```

Quando un livello viene completato, viene rimosso per liberare la memoria dinamicamente, collegando i livelli adiacenti per non interrompere la catena.

3. Collisioni come "Single Source of Truth"

Un'altra sfida importante era la verifica delle collisioni. Dato che l'area di gioco cambia spesso (le bombe distruggono le casce e liberano lo spazio), inizialmente avevo pensato di usare una seconda matrice di interi per tracciare gli ostacoli. Questo metodo era inefficiente e portava facilmente a errori nel codice. La mia soluzione definitiva sfrutta direttamente la matrice visiva in `char` (quella disegnata a schermo) come **Single Source of Truth**, ovvero come unico punto di riferimento vero per l'intero gioco. Tramite la funzione `getCella(y,x)` controllo i bordi dell'array e comunico in modo sicuro alle altre classi se una coordinata è libera.

4. Sistema di Power-Up (Items)

Per attivare gli effetti degli oggetti senza complicare il lavoro del Gamemaster, ho sfruttato le basi del C++. L'oggetto `Item` usa il suo metodo `apply(Player* p)` per inserire direttamente le modifiche sulle statistiche del giocatore, permettendo al codice di restare pulito e ordinato.

4.2 Utilizzo degli LLM per Ottimizzare il Codice

Durante lo sviluppo, ho usato l'Intelligenza Artificiale come supporto alla scrittura, concentrandomi su due problemi: **risoluzione di errori in memoria** causati da una cattiva impostazione delle matrici e **miglioramento del rendering visivo** nel terminale.

2.1 Caso d'Uso 1: Rendering Sicuro e Risoluzione Errori di Matrice (Mappa.cpp)

Obiettivo: Prevenire errori visivi e ottimizzare l'uso della memoria nella mappa del gioco.

Prompt inviato all'LLM:

"In C++ con ncurses, ho creato una matrice 40x20 per la mappa ma la stampa a schermo presenta bug. Uso ACS_BLOCK per i muri ma a volte compaiono caratteri estranei. Il codice è: `char griglia[40][20];`. Come posso migliorare la grafica e risolvere i problemi in memoria?"

Risposta dell'LLM:

"Il problema principale è l'allineamento della matrice. In C++ gli array seguono l'ordine `[righe][colonne]`. Dato che il terminale è largo (40 caratteri) e basso (20 caratteri), stai dichiarando 40 righe e 20 colonne. Quando scorri per 40 passi in orizzontale, leggi memoria sbagliata causando errori! Modifica in `griglia[20][40]`. Inoltre, per i muri, stampa il carattere SPAZIO e colora lo sfondo con `COLOR_PAIR` invece di usare `ACS_BLOCK`."

Codice Prima dell'Ottimizzazione:

```
// Errore di allineamento (Righe vs Colonne)
char griglia[40][20];

void Mappa::disegnaLayout() {
    for (int r=0; r<40; r++) {
        for (int c=0; c<20; c++) {
            if (griglia[r][c] == '#') {
                // Instabile su alcuni emulatori
                addch(ACS_BLOCK);
            } else if (griglia[r][c] == '%') {
                addch(ACS_BOARD);
            }
        }
    }
}
```

Codice Finale Ottimizzato (src/Mappa.cpp):

```
// Allineamento matriciale corretto
char griglia[20][40];

void Mappa::disegnaLayout() {
    for(int riga=0; riga<altezza; riga++) {
        for(int col=0; col<larghezza; col++) {
            char blocco = griglia[riga][col];
            if (blocco == '%') {
                attron(COLOR_PAIR(4) | A_DIM);
                addch(ACS_CKBOARD);
                attroff(COLOR_PAIR(4) | A_DIM);
            } else if (blocco == '#') {
                attron(COLOR_PAIR(5)); // Sfondo
                addch(' ');           // Spazio
                attroff(COLOR_PAIR(5));
            } else { addch(' '); }
        }
    }
}
```

Differenze e Valutazione Critica: Invertire le dimensioni in `griglia[20][40]` ha eliminato subito i comportamenti anomali del gioco, che andava a leggere memoria fuori dai confini. Il suggerimento dell'LLM di colorare uno spazio vuoto con `COLOR_PAIR(5)` per simulare il muro pieno garantisce una grafica pulita su qualunque terminale si avvii il gioco.

2.2 Caso d'Uso 2: Controllo Collisioni con Single Source of Truth (Mappa.cpp)

Obiettivo: Semplificare il codice per le collisioni, dimezzando il consumo di RAM ed evitando errori di aggiornamento delle variabili.

Prompt inviato all'LLM:

"Per bloccare il movimento, ho creato una seconda matrice parallela `int collisioni[20][40]` riempita con 1 (ostacolo) e 0. Faccio molta fatica a tenerla aggiornata quando le bombe rompono le casse, perché devo cambiare le coordinate in più file. C'è un modo migliore per farlo in C++?"

Risposta dell'LLM:

"Sì, avere due matrici crea ridondanza e porta a errori. Devi avere un unico punto di riferimento per i dati, noto come **Single Source of Truth**: la tua matrice visiva in `char` contiene già le informazioni necessarie per capire se c'è un ostacolo. Crea un metodo `getCella()` che legge direttamente dallo schermo virtuale."

Codice Prima dell'Ottimizzazione:

```
// Costo O(N*M) per tenere le matrici
// sincronizzate
int matrice_collisioni[20][40];

bool Mappa::checkCollision(int y, int x) {
    // Rischio crash se x,y fuori dalla
    // mappa
    if (matrice_collisioni[y][x] == 1) {
        return true; // Ostacolo trovato
    }
    return false;
}
```

Codice Finale Ottimizzato (src/Mappa.cpp):

```
// Usa char griglia[20][40] come
// riferimento. Costo: O(1) costante
char Mappa::getCella(int y, int x) {
    // Controllo sicurezza sui bordi per
    // evitare Segmentation Fault
    if (y >= 0 && y < altezza &&
        x >= 0 && x < larghezza) {

        return griglia[y][x];
        // Torna '#', '%' o ' ' in tempo
        // reale
    }
    return '#'; // Oltre il confine e' muro
    !
}
```

Differenze e Valutazione Critica: Questa ottimizzazione ha semplificato notevolmente il mio lavoro. Togliendo la seconda matrice intera ho risparmiato memoria (passando da una complessità spaziale $O(2N)$ a una ottimale $O(N)$), ma soprattutto ho eliminato i fastidiosi "falsi muri" invisibili che capitavano quando la matrice si desincronizzava. A livello di costo computazionale (temporale), l'algoritmo di verifica resta $O(1)$, ma abbiamo completamente azzerato il ciclo di operazioni prima necessario per tenere sincronizzate due matrici diverse ad ogni esplosione.

5. PARTE 3: THOMAS GHINI - Menù, Classifica, Tempo e Responsività

5.1 Principali Sfide Tecniche e Come Sono State Affrontate

La mia area di competenza copriva le schermate esterne (Menù, Game Over), la gestione del timer visivo di gioco, l'interfaccia grafica (adattamento della finestra) e l'interazione con la lettura e la scrittura dei file di testo (la Classifica).

1. Menù Principale e Posizionamento Dinamico (Responsive)

Ho creato la classe `Menu`, che disegna il titolo "BOMBERMAN" e tre bottoni cliccabili col mouse: Start, Classifica e Exit. Il primo problema che ho incontrato è stato il posizionamento dei bottoni: scrivendo coordinate fisse, appena l'utente allargava la finestra i bottoni si accavallavano. Ho risolto chiedendo direttamente alla libreria la dimensione della finestra attuale, così i bottoni vengono ricalcolati dinamicamente.

2. Sistema di Classifica con Lettura dei File

La classifica deve accettare input da tastiera, ma `ncurses` non è fatto per acquisire frasi intere. Ho dovuto costruire un ciclo manuale per permettere di scrivere e cancellare (usando il tasto `BACKSPACE`). Inoltre, dato che i record salvati superano spesso lo spazio visibile, ho inventato uno scorrimento (uno scroll) in cui il programma salta alcune righe usando `file.seekg(0)` e `file.getline()`, mostrando solo la parte della classifica che interessa all'utente.

3. Responsività Globale (Classe Schermo)

Il gioco doveva funzionare bene sui terminali di tutti i professori. Il metodo `controllaDimensione()` mette in pausa il gioco mostrando il messaggio "Finestra troppo piccola!", bloccando l'esecuzione finché l'utente non la riallarga a sufficienza.

```
void Gamemaster::responsive() {  
    int h, w; getmaxyx(stdscr, h, w);  
    if (h != oldH || w != oldW) { /* logic */ }  
}
```

4. Gestione Tempo con Pause Involontarie

Il timer conta 240 secondi per livello. Il problema era che, mettendo il gioco in pausa per una finestra troppo piccola, il timer "reale" continuava a scendere. Ho aggiunto una funzione `addpausa()` che salva il tempo sprecato e lo restituisce al giocatore una volta ripresa la partita.

5.2 Utilizzo degli LLM per Ottimizzare il Codice

La gestione dell'input da tastiera sul terminale ha causato alcuni bug legati alle funzioni video di `ncurses`. L'IA mi è stata di grandissimo aiuto per capire come prevenire gli errori in C++ e scoprire funzioni poco note di `ncurses`, rendendo molto più stabile la lettura e scrittura dei file.

3.1 Caso d'Uso 1: Creazione e Lettura Sicura del File Classifica

Obiettivo: Prevenire la chiusura forzata del gioco (crash) se il file di testo manca al primo avvio.

Prompt inviato all'LLM:

"Ho un programma in C++ che legge da un file di testo la classifica. Il problema è che se il file `Classifica.txt` non c'è, il programma si blocca e va in errore perché prova a leggere il vuoto. Come risolvo in C++?"

Risposta dell'LLM:

"Usa un approccio sicuro per i file: prima di iniziare a leggere, usa la funzione `is_open()` per assicurarti che l'apertura abbia avuto successo. Se non si è aperto, crealo temporaneamente con `ofstream`, chiudilo in modo da svuotare il buffer di salvataggio, e poi riaprilo. Infine, gestisci il caso in cui ci sia un errore grave (come mancanza di permessi sul PC)."

Codice Prima dell'Ottimizzazione:

```
void Classifica::leggiFile() {
    ifstream file("Classifica.txt");
    int cha;

    // Crasha o si blocca se file
    inesistente
    // Manca un controllo preventivo
    box(wClass, 0, 0);
    // ... lettura cieca da 'file'
}
```

Codice Dopo (src/Classifica.cpp):

```
// Inizio metodo leggiFile
ifstream file("Classifica.txt");
int cha;

if (!file.is_open()) {
    ofstream NuovoFile("Classifica.txt");
    NuovoFile.close();
    file.open("Classifica.txt");
}

if (!file.is_open()) {
    box(wClass, 0, 0);
    mvwprintw(wClass, 3, 2, "Errore!");
    wrefresh(wClass);
    // Loop gestione Errore...
}
```

Differenze e Valutazione Critica: Questa aggiunta rende il gioco molto più sicuro. Ora la classifica si crea da sola se non esiste; se invece il file non può essere creato (ad esempio per mancanza di permessi nella cartella), il programma non va in crash, ma mostra un semplice messaggio di "Errore!" e continua a funzionare senza bloccarsi improvvisamente.

3.2 Caso d'Uso 2: Pulizia dell'Input in eccesso (Classifica.cpp)

Obiettivo: Evitare l'inserimento di "caratteri fantasma" nel campo di testo, causati dal mouse o dai tasti premuti prima.

Prompt inviato all'LLM:

"Quando l'utente clicca su 'Classifica' nel menù, si apre il campo per inserire il numero. A volte il campo si riempie già da solo con le frecce premute in ritardo nel menù. Come posso svuotare questo input memorizzato prima di fare la lettura vera e propria?"

Risposta dell'LLM:

"La libreria ncurses conserva un archivio dei tasti premuti. Per svuotarlo e iniziare a leggere in modo pulito, usa la funzione `flushinp()` all'inizio del tuo ciclo di lettura. A differenza della funzione classica del C `fflush(stdin)`, questa funzione annulla i tasti pendenti in modo affidabile, senza causare errori inaspettati nel programma."

Codice Prima (Con Glitch Tasti):

```
void Classifica::Npunti() {
    // Inizia la finestra di Input
    bool inInput = true;
    while (inInput) {
        if (!sch.controllaDimensione(40,20)
    ) {
            werase(stdscr);
            refresh();
        }
        // Rischio di prendere l'ultimo
        // premuto per sbaglio nel menu!
        int tast = wgetch(win);
        // ...
    }
```

Codice Dopo (src/Classifica.cpp):

```
void Classifica::Npunti() {
    flushinp(); // Svuota buffer ncurses
    bool inInput = true;
    while (inInput) {
        if (!sch.controllaDimensione(40,20)
    ) {
            werase(stdscr);
            // Forza il ridisegno
            touchwin(stdscr);
            refresh();
        }
        int tast = wgetch(win);
        // ...
    }
```

Differenze e Valutazione Critica: Questa semplice istruzione a riga singola mi ha permesso di risolvere un fastidioso bug di input che peggiorava molto l'esperienza dell'utente. Inoltre, nel codice "Dopo" ho inserito un'altra piccola ottimizzazione suggerita dall'IA: la funzione `touchwin(stdscr)`. Questa istruzione dice a ncurses che l'intero schermo deve essere forzatamente aggiornato durante un ridimensionamento, evitando così l'effetto "scia" o la duplicazione dei bordi della finestra.